

Relatório Técnico

LIACD TP1

From Raw Detections to Real Intelligence

1. Introdução

Este trabalho apresenta a implementação de um pipeline de *retail intelligence* organizado em duas fases. A primeira fase reconstrói trajetórias individuais a partir de eventos anónimos gerados por sistemas de visão computacional. A segunda fase transforma essas trajetórias em métricas, *insights* acionáveis e num briefing semanal dirigido ao gestor de loja, recorrendo a um LLM local através do Ollama.

O principal desafio da Fase 1 está no facto de o ficheiro `events.csv` não conter `person_id`. Os eventos de entrada, permanência e saída surgem intercalados no tempo, podem corresponder a várias pessoas em simultâneo e incluem atributos demográficos estimados, naturalmente sujeitos a erro. Assim, o sistema tem de resolver um problema de *data association*: associar observações anónimas ao visitante mais provável que as originou.

Na Fase 2, foi respeitada a separação definida no enunciado: o LLM não acede ao CSV nem calcula métricas diretamente sobre dados brutos. Toda a computação é realizada em Python, primeiro no `stitcher.py` e depois no `analytics.py`. O LLM recebe apenas métricas previamente calculadas e rascunhos factuais derivados dessas métricas. A sua função é linguística: sintetizar, redigir e tornar os resultados mais claros e acionáveis.

Esta decisão também está alinhada com os conteúdos lecionados. A Aula 6 defende a avaliação sistemática, o uso de *ground truth* sempre que possível, métricas e análise de erro. A Aula 7 alerta para enviesamentos de *prompt*, em particular *format priming*, linguagem vaga e *anchoring*. A Aula 8 mostra que modelos com contexto longo podem perder informação relevante no meio do *prompt*. Por esse motivo, o sistema reduz os dados a métricas compactas e aplica validação pós-LLM.

2. Algoritmo de Stitching

O módulo `src/stitcher.py` recebe `events.csv` e produz `journeys.csv`. O algoritmo processa os eventos por ordem temporal e mantém três estruturas principais:

1. `active_in_zone`: *tracks* atualmente abertas dentro de cada zona.
2. `idle_tracks`: *tracks* que saíram de uma zona e que ainda podem ser associadas a uma entrada posterior.
3. `all_tracks`: *tracks* finalizadas.

Cada *track* guarda as visitas a zonas, o estado atual, a última zona visitada, o último timestamp de saída e os votos acumulados para género e faixa etária.

Heurísticas usadas

1. Ordenação por timestamp

Os `event_id` não são usados como ordem global, porque podem refletir a origem do sensor. O timestamp é a única âncora temporal comum e, por isso, é usado como referência principal.

2. Janela de 5 minutos

Uma *track* só pode ser associada a outra zona se o intervalo entre o `exit` e o novo `entry` for inferior ou igual a 300 segundos. Este valor vem diretamente do enunciado.

3. Plausibilidade física

Quando o ficheiro `zones.json` está disponível, o sistema pré-calcula os menores tempos de caminhada entre todos os pares de zonas usando Dijkstra. Uma transição é rejeitada se for fisicamente impossível dentro do intervalo observado, considerando uma tolerância de 20 segundos.

4. Consistência de atributos com tolerância ao erro

O género e a faixa etária são tratados através de votos. Em *tracks* com poucas observações, um erro de classificação não rejeita automaticamente a associação. Já em *tracks* com evidência suficiente, um género incompatível leva à rejeição da associação. Esta abordagem reflete os erros de classificação mencionados no enunciado.

5. Estrutura `entry -> linger -> exit`

Os eventos `linger` e `exit` são associados a *tracks* ativas na mesma zona. Em caso de empate, é usada a estratégia FIFO, assumindo que quem entrou primeiro tende a sair primeiro.

6. Entradas em `Z_E`

As entradas em zonas de porta, como `Z_E1` e `Z_E2`, iniciam uma nova *track*. Isto evita associar uma pessoa que já está dentro da loja a um novo evento de entrada pela porta.

Output da Fase 1

O journeys.csv gerado contém as 9 colunas exigidas:

person_id, zone_id, entry_time, exit_time, dwell_s, gender, age_range, visit_date, hour_of_day

Cada linha representa a visita de uma pessoa sintética a uma zona.

Qualidade observada

Nos outputs atuais, o journeys.csv tem 85.347 linhas e 11.051 pessoas sintéticas. Na avaliação sobre events_validation.csv, o harness registou os seguintes resultados:

Métrica	Valor
consistência	100.0%
Cobertura	100.0%
Eventos atribuídos	250.000 / 250.012
Completude	51.17%
Gap medio	40.6s
Gap mediano	24.0s
Gap p95	149.0s
Gaps > 300s	0.0%
Gaps negativos	0.0%

A completude é inferior à consistência e à cobertura, mas isso é esperado. Parte das trajetórias pode começar ou terminar fora da janela temporal capturada, e algumas saídas podem não ter sido observadas. O valor mais importante para evitar erros graves é a consistência temporal, que ficou em 100%.

3. Pipeline Analítico

O módulo src/analytics.py recebe journeys.csv e produz metrics.json. Todas as métricas pedidas no enunciado são calculadas em Python antes de qualquer chamada ao LLM.

Métricas calculadas

1. Tráfego

São calculados os visitantes únicos por dia, os visitantes únicos por hora, o total semanal, a duração média e mediana da visita, o dia mais movimentado, o dia mais calmo e a hora de pico.

2. Zonas

São calculadas as visitas por zona, o dwell médio e mediano, a taxa de paragem e as 10 principais sequências de zonas.

3. Funil

São calculados os visitantes totais, os visitantes com entrada registada, os visitantes que chegaram à caixa, a taxa de conversão, o perfil dos visitantes que não chegaram à caixa e o alcance por zona.

4. Demografia

São calculadas distribuições de género por hora, faixa etária por hora, *dwell* médio por género/zona e *dwell* médio por faixa etária/zona.

5. Anomalias

É criado um *baseline* com os primeiros 6 dias, comparado com o dia 7, e são assinaladas anomalias com desvio superior a 2 sigma.

Valores principais no dataset de treino

Item	Valor
Visitantes semanais	11.051
Dia mais movimentado	2025-03-15, com 1.964 visitantes
Dia mais calmo	2025-03-11, com 1.249 visitantes
Hora de pico	17h
Duração media	15.3 minutos
Chegaram a caixa	8.904
Taxa de conversão	80.6%
Não chegaram a caixa	2.147, ou 19.4%
Total de anomalias no treino	28

As datas em `baseline_days` foram normalizadas para o formato YYYY-MM-DD. Também foi acrescentado no `checkout_rate_pct` ao `metrics.json`, para evitar que o LLM tivesse de calcular $100 - 80.6$.

4. Engenharia de Prompts

O módulo `src/insights.py` implementa as duas estratégias obrigatórias:

1. Estratégia A: *zero-shot*.
2. Estratégia B: *few-shot*.

Ambas usam o mesmo schema JSON pedido no enunciado e recebem apenas uma versão compacta do `metrics.json`.

Estrategia A — Zero-shot

O *prompt zero-shot* inclui uma persona de analista sénior de retalho, dados agregados da loja, o schema JSON, regras contra números inventados, a proibição de frases vagas como “pode indicar”, “possivelmente” e “melhorar o atendimento”, e a instrução para responder apenas com JSON.

Esta estratégia funciona como *baseline*, em linha com a Aula 6: primeiro mede-se o comportamento base antes de introduzir exemplos.

Estrategia B — Few-shot

O *prompt few-shot* acrescenta exemplos de mau e bom *insight*, seguindo a tabela do enunciado:

1. Mau: *insight* genérico.
2. Bom: *insight* com zona, período, número concreto e ação operacional.

Esta abordagem está alinhada com a Aula 7. Os exemplos ajudam a reduzir a vagueza e a formatar melhor a resposta, mas também podem introduzir risco de *anchoring*. Por isso, os exemplos foram mantidos curtos e a resposta final passa por validação.

Validacao pos-LLM

Depois da resposta do LLM, o insights.py aplica uma camada de validação que inclui:

1. De duplicação por título e observação normalizados.
2. Normalização de categorias e urgências.
3. Remoção de frases vagas proibidas.
4. Criação de âncoras factuais em Python a partir do metrics.json.
5. Prioridade a *insights* factuais quando há conflito com o output bruto do LLM.

Esta camada aplica diretamente o que foi lecionado sobre avaliação de LLMs: o *prompt* por si só não é suficiente; é necessário medir, verificar e limitar o domínio factual.

Comparação quantitativa A vs B

O insights.json guarda a comparação entre as duas estratégias. No output atual, os resultados são:

Métrica	Zero-shot	Few-shot
Insights gerados	10	10
Categorias cobertas	4	5
Duplicados	0.0%	0.0%

Métrica	Zero-shot	Few-shot
Observações com ≥ 2 números	60.0%	70.0%
Score	62.0	66.5
Vencedor	—	Few-shot

O resultado bruto do LLM ainda apresenta algumas fragilidades, incluindo urgências fora do schema e frases como “pode indicar”. Isto é esperado num modelo local pequeno. O ficheiro final não guarda esses outputs brutos; guarda apenas a comparação quantitativa e os 10 *insights* validados.

Verificação local dos insights finais

- 10 insights finais
- IDs de INS_001 a INS_010
- 5 categorias cobertas
- 0 duplicados
- 0 ocorrências de "pode indicar", "possivelmente" ou "melhorar o atendimento"
- Urgências dentro do conjunto pedido: imediata, esta semana, próximo mês

Report semanal

O módulo `src/report.py` gera `weekly_report.md`. Na versão final, o *report* volta a usar o LLM local, como pedido no enunciado. A diferença é que o LLM não recebe dados brutos nem liberdade para criar factos. Recebe rascunhos factuais calculados em Python e redige cada secção com base neles.

O processo é:

- Python constrói rascunhos factuais para as 6 secções do report.
- O LLM reescreve cada rascunho em linguagem de gestor.
- Uma guarda simples verifica se o LLM introduziu números que não estavam no rascunho.
- Se houver números novos, a secção volta ao rascunho factual.

Esta decisão segue a Aula 8, ao reduzir o contexto e manter os factos relevantes próximos da tarefa. Também segue a Aula 6, ao aplicar validação automática ao output.

O report final contém as 6 secções pedidas:

- Resumo Executivo
- Performance de Tráfego
- Análise de Zonas
- Funil de Clientes
- Anomalias da Semana
- Recomendações para a Próxima Semana

5. Harness de Avaliação

O ficheiro `evaluate.py` executa o pipeline de ponta a ponta com um único comando:

```
python evaluate.py --data data/events_validation.csv --output  
output/evaluation_report.json --ground-truth data/ground_truth.json
```

O *harness* realiza as seguintes etapas:

- Corre o stitcher.
- Corre o analytics.
- Corre o insight com a estratégia B.
- Corre o report.
- Calcula métricas da Fase 1.
- Calcula precisão numérica, ausência de alucinação e deteção de anomalias

Última avaliação disponível

A última avaliação guardada em `output/evaluation_report.json` apresenta:

Métrica	Valor
Pipeline status	OK
consistência	100.0%
Cobertura	100.0%
Compleitude	51.17%
Precisão numérica dos insights	72.6%
Ausência de alucinação no report	89.5%
Deteção de anomalias	75.0% (3/4)

O detalhe da deteção de anomalias é o seguinte:

Anomalia	Resultado
ANO_001 — Z_N1 as 10h	Detectada
ANO_002 — Z_S5 as 14h	Detectada
ANO_003 — Z_N7 as 11h	Nao detectada
ANO_003 — Z_N7 as 12h	Detectada

Estes resultados mostram que a Fase 1 está muito robusta, mas que a Fase 2 ainda é sensível à forma como as anomalias são seleccionadas e verbalizadas. A validação final já foi corrida com LLM local. O *report* acionou o *guardrail* de números numa secção, substituindo a redação do LLM pelo rascunho factual para evitar alucinação numérica.

6. Validação Face ao Enunciado

Pontos cumpridos

O projeto cumpre os principais requisitos definidos no enunciado:

1. Existem os quatro módulos principais: `stitcher.py`, `analytics.py`, `insights.py` e `report.py`.
2. Cada módulo é executável por linha de comandos.
3. O `journeys.csv` tem o schema pedido.
4. O `metrics.json` contém as métricas pedidas.
5. O `insights.py` implementa *zero-shot* e *few-shot*.
6. O `insights.json` contém os campos obrigatórios.
7. O `report.py` gera Markdown com as 6 secções pedidas.
8. O `evaluate.py` testa o pipeline de ponta a ponta.
9. O LLM usado é local via Ollama (`llama3.1:8b`).
10. A temperatura é 0 e a seed é fixa, aumentando a reprodutibilidade.
11. Existem *prompts* versionados em `prompts/`.
12. Existem `inject_anomalies.py` e `ground_truth.json` para cenários próprios.

Pontos a defender oralmente

1. Report com `--metrics` explícito

O *report* pode receber `--metrics` explicitamente para reforçar o *grounding* factual. Se o avaliador exigir a forma minimalista do enunciado, o comando `report.py --input output/insights.json --output output/weekly_report.md` continua a funcionar usando apenas os *insights*.

2. Campos extra em `insights.json`

O output final de `insights.json` inclui campos extra, como `comparacao_estrategias` e resultados brutos das estratégias. O schema pedido está presente em `insights`, mas foram mantidos metadados para justificar a comparação quantitativa A vs B.

3. Detecção textual de anomalias

A deteção de anomalias no *harness* é textual: procura zona e hora nos *insights*. Isto pode penalizar respostas semanticamente válidas que não mencionem a hora exata no formato esperado.

7. Código Face ao Lecionado

O projeto está alinhado com os principais conceitos abordados nas aulas.

Aula 6 — Prompt Evaluation

- Existem métricas automáticas.

- Existe comparação entre estratégias.
- Existe *ground truth* para anomalias injetadas.
- Existe análise de falhas.
- Existe validação pós-LLM, não apenas *prompting*.

Aula 7 — Prompt Bias

- O sistema reconhece o risco de *format priming*: pedir exatamente N *insights* pode levar a repetição.
- O *prompt* evita linguagem vaga.
- O output final remove frases proibidas.
- A comparação A/B mede os efeitos do *few-shot*.
- A camada de âncoras reduz o risco de *anchoring* nos exemplos *few-shot*. **Aula 8 — Contexto e RAG/Grounding**
- O LLM não recebe CSV nem contexto excessivo.
- Os factos são compactados e estruturados.
- O *report* usa rascunhos factuais.
- A verificação de números reduz alucinação.
- A arquitetura aproxima-se de uma lógica de *retrieval/grounding*: Python fornece evidências e o LLM sintetiza.

8. Limitações Honestamente Identificadas

Apesar dos bons resultados, existem limitações importantes:

- A completude das trajetórias é apenas cerca de 51%, provavelmente devido a truncagem temporal e saídas não capturadas.
- A deteção de anomalias ainda não atingiu 100% na última avaliação guardada. Esta é a principal limitação atual da Fase 2.
- Os outputs brutos das estratégias A e B ainda podem conter frases proibidas ou urgências fora do schema. Por isso, o ficheiro final guarda apenas a comparação quantitativa e não inclui os textos brutos.
- O modelo llama3.1:8b é pequeno e pode falhar em contexto longo, cálculo simples ou seleção das anomalias mais relevantes.
- A métrica de ausência de alucinação é baseada em números verificáveis. Pode penalizar texto correto quando o número aparece com uma formatação diferente.
- O *report* é melhorado por LLM, mas alguns termos técnicos podem manter-se se estiverem no rascunho factual. Por esse motivo, foi adicionada uma regra para evitar “sigma” e “desvio padrão”.

9. Conclusão

A Fase 1 está sólida e cumpre bem o objetivo do enunciado. O algoritmo é eficiente, usa heurísticas justificadas e atinge consistência e cobertura de 100% na avaliação.

A Fase 2 cumpre a arquitetura pedida: Python calcula, o LLM sintetiza. A solução final é mais robusta do que uma abordagem baseada apenas em *prompting*, porque usa âncoras factuais e validação pós-LLM. Isto está alinhado com os conteúdos das aulas, onde a avaliação e a mitigação de alucinações são tratadas como partes centrais do desenvolvimento.

O ponto mais fraco continua a ser a deteção textual de anomalias no *harness*. O sistema já deteta anomalias no *metrics.json*, mas nem sempre as transforma em *insights* com zona e hora exatamente no formato esperado. A melhoria futura mais importante seria adicionar campos estruturados aos *insights* de anomalia, como *zone_id*, *hour*, *observed_value* e *baseline_mean*, para que a avaliação não dependa de *matching* textual.

Anexos

- Prompts completos: *prompts/insights_zero_shot_v1.txt*, *prompts/insights_few_shot_v1.txt*, *prompts/report_v1.txt*
- Exemplo de metricas: *output/metrics.json*
- Exemplo de insights: *output/insights.json*
- Briefing semanal: *output/weekly_report.md*
- Avaliacao: *output/evaluation_report.json*